

UNIVERSIDAD DEL VALLE DE GUATEMALA



Proyecto 2

Andre Marroquin Tarot - 22266

Sergio Orellana - 221122

Computación paralela y distribuida

Parte A:

DES Proceso de cifrado y descifrado

Datos de entrada (tamaños)

- **Bloque de datos:** 64 bits.
- **Clave:** 64 bits (56 útiles + 8 de paridad).
- **Subclaves:** 16 subclaves $K_1..K_{16}K_1..K_{16}K_1..K_{16}$ de 48 bits.

Glosario

- **PC-1:** Elección Permutada 1. Quita paridad y reordena la clave (64→56).
- **C0, D0:** Mitades de 28 bits que resultan de PC-1 (izq./der.).
- **PC-2:** Elección Permutada 2. Selecciona/reordena 48 bits de $C_i || D_i \rightarrow K_i$
- **IP:** Permutación inicial al bloque de datos.
- **IP⁻¹:** Inversa de IP (permuta final).
- **L0, R0 / Li, Ri:** Mitades izquierda/derecha del bloque tras IP y tras la ronda i.
- **E(R):** Expansión de 32→48 bits para poder XORear con $K_iK_{-i}K_i$.
- **S-Boxes:** 8 tablas que convierten 6→4 bits (en total 48→32).
- **P:** Permutación que reordena los 32 bits que salen de las S-Boxes.
- **f(R,K):** Función de ronda = $P(S(E(R) \oplus K))$.
- $\oplus$: XOR (o exclusivo).
- $||$: Concatenación de bits.

Cifrado (plaintext → ciphertext)

1. **Key schedule**
 - 1.1 PC-1: clave 64→56; dividir en C0 y D0 (28/28).
 - 1.2 Para $i=1..16$: rotar C_{i-1}, D_{i-1} (1 o 2 bits) → C_i, D_i .
 - 1.3 PC-2 sobre $C_i || D_i \rightarrow K_i$ (48 bits).
2. **IP** al bloque de 64 bits.
3. **Dividir** en L0 y R0 (32/32).
4. **Rondas Feistel (i=1..16):**
 - 4.1 $X = E(R_{i-1}) \oplus K_i$ (32 → 48, XOR) .
 - 4.2 Pasar X por S-Boxes → 32 bits.

4.3 P sobre esos 32 bits.

4.4 **Actualizar mitades:**

- $L_i = R_{i-1}$

- $R_i = L_{i-1} \oplus f(R_{i-1}, K_i)$.

5. **Intercambio final:** formar $R_{16} || L_{16}$.

6. $IP^{-1} \rightarrow$ bloque cifrado de 64 bits.

Descifrado (ciphertext $\rightarrow$ plaintext)

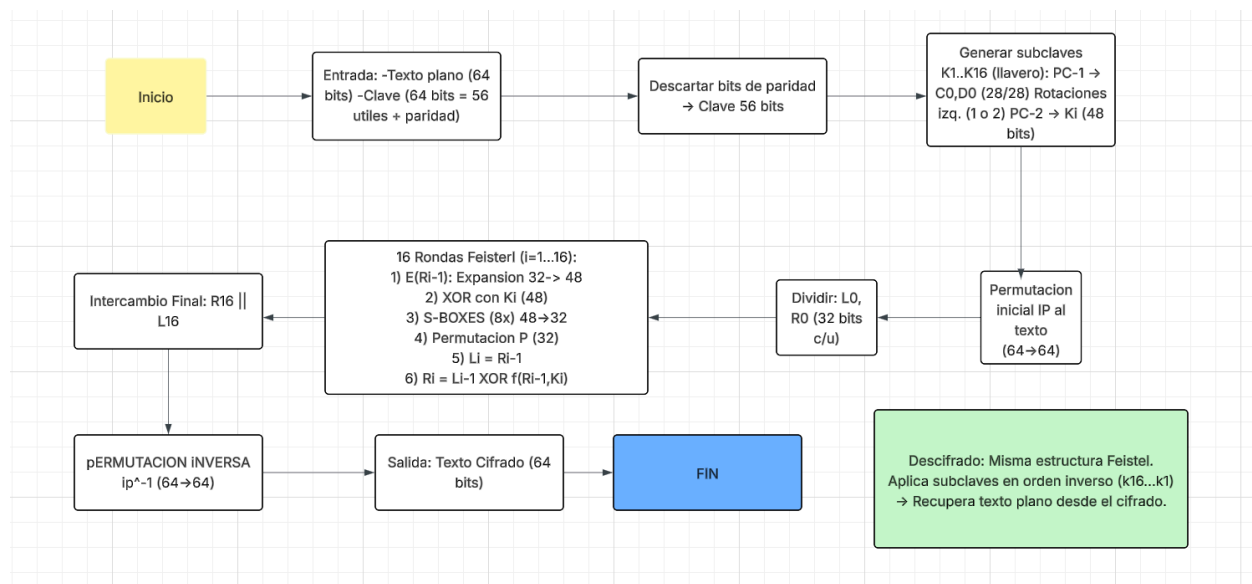
1. IP al bloque cifrado; dividir en L_0, R_0 .

2. **Rondas Feistel ($i=16..1$):** repetir los pasos 4.1–4.4 pero usando las subclaves en orden **inverso**: $K_{16}, K_{15}, \dots, K_1$.

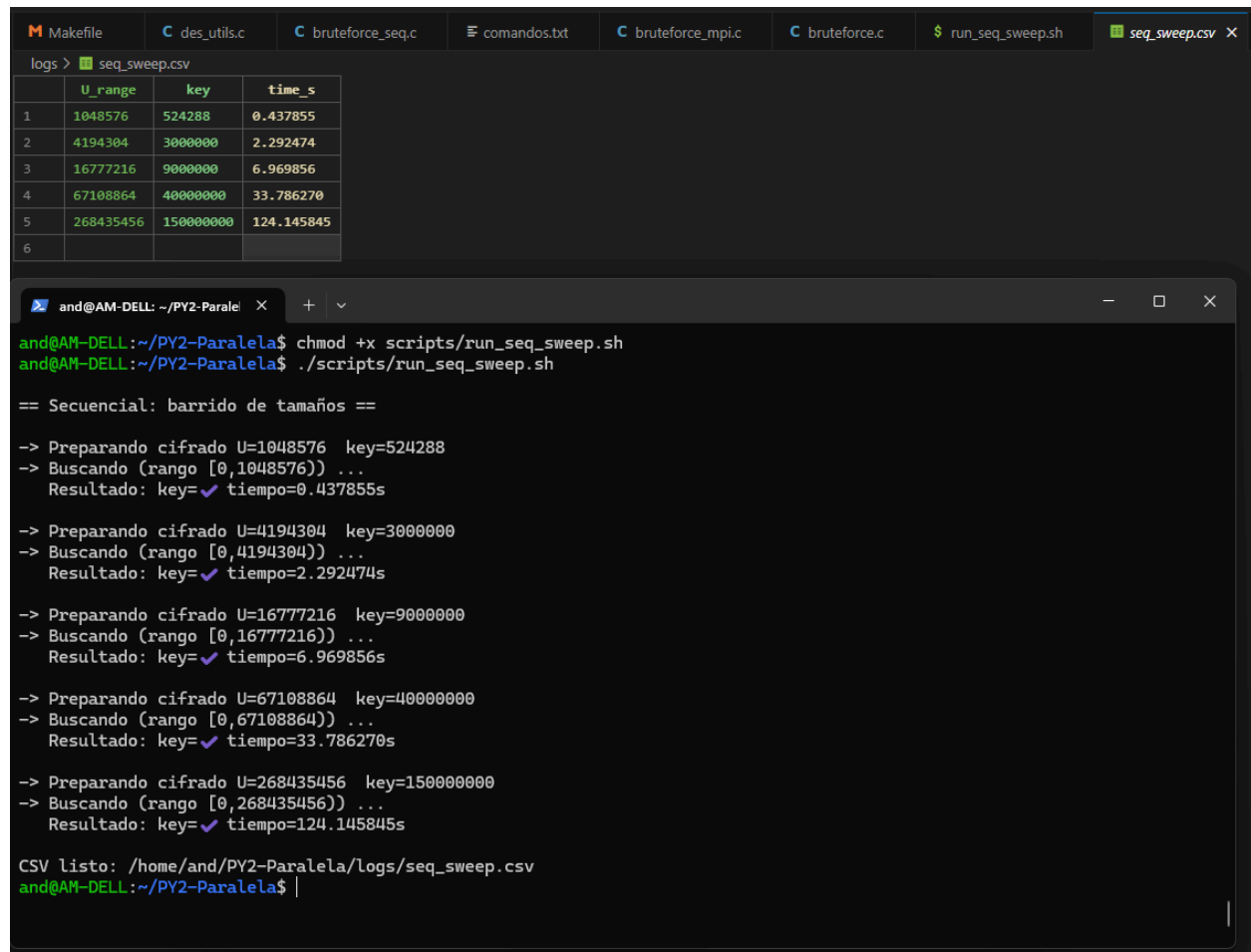
3. **Intercambio final:** $R_{16} || L_{16}$.

4. $IP^{-1} \rightarrow$ texto plano original.

Diagrama de Flujo



Inciso 3



```
logs > seq_sweep.csv
```

	U_range	key	time s
1	1048576	524288	0.437855
2	4194304	3000000	2.292474
3	16777216	9000000	6.969856
4	67108864	40000000	33.786270
5	268435456	150000000	124.145845
6			

```
and@AM-DELL: ~/PY2-Paralela$ chmod +x scripts/run_seq_sweep.sh
and@AM-DELL:~/PY2-Paralela$ ./scripts/run_seq_sweep.sh

== Secuencial: barrido de tamaños ==

-> Preparando cifrado U=1048576 key=524288
-> Buscando (rango [0,1048576)) ...
Resultado: key=✓ tiempo=0.437855s

-> Preparando cifrado U=4194304 key=3000000
-> Buscando (rango [0,4194304)) ...
Resultado: key=✓ tiempo=2.292474s

-> Preparando cifrado U=16777216 key=9000000
-> Buscando (rango [0,16777216)) ...
Resultado: key=✓ tiempo=6.969856s

-> Preparando cifrado U=67108864 key=40000000
-> Buscando (rango [0,67108864)) ...
Resultado: key=✓ tiempo=33.786270s

-> Preparando cifrado U=268435456 key=150000000
-> Buscando (rango [0,268435456)) ...
Resultado: key=✓ tiempo=124.145845s

CSV listo: /home/and/PY2-Paralela/logs/seq_sweep.csv
and@AM-DELL:~/PY2-Paralela$ |
```

En las pruebas secuenciales, al aumentar el tamaño del espacio de llaves (U_range) el tiempo de búsqueda creció prácticamente de forma lineal, tal como esperaría un ataque de fuerza bruta clásico. Por ejemplo, al aumentar el rango varias veces, el tiempo se multiplicó entre $\sim 3\times$ y $\sim 5\times$ por variaciones normales del sistema, pero el rendimiento por segundo se mantuvo muy estable alrededor de 1.2–1.3 millones de llaves por segundo (ejemplo 524 288 llaves en 0.438 s $\rightarrow \approx 1.20$ M/s; 3 000 000 en 2.29 s $\rightarrow \approx 1.31$ M/s; 150 000 000 en 124.1 s $\rightarrow \approx 1.21$ M/s). Dado que las llaves probadas están cerca de la mitad del espacio, el número promedio de intentos es $\approx U/2$, por lo que el tiempo total escala con U.

Inciso 4:

Se hizo funcionar el bruteforce.c

```

and@AM-DELL: ~/PY2-Paralela$ mpirun -np 4 ./bin/bruteforce_orig -s "the" -L 0 -U 16777216
=====
BruteDES • MPI (archivo original adaptado)
- Tabla por proceso + resumen global
=====

→ BRUTEFORCE MPI (original adaptado)
• Procesos : 4
• Subcadena: "the"
• Rango : [0, 16777216)

• Detalle por proceso
RANK | START | END | TESTS | STATUS | TIME(s)
-----|-----|-----|-----|-----|-----
0 | 0 | 4194304 | 105735 | FOUND | 0.0650
1 | 4194304 | 8388608 | 103410 | STOP(SIGNAL) | 0.0651
2 | 8388608 | 12582912 | 105394 | STOP(SIGNAL) | 0.0651
3 | 12582912 | 16777216 | 82531 | STOP(SIGNAL) | 0.0650

• Resultado: ✓ Llave encontrada
- Rank : 0
- Llave : 105734
- Texto : Save the planet

• Resumen global
- Llaves probadas totales : 397070
- Tiempo total (max rank) : 0.065147 s

and@AM-DELL:~/PY2-Paralela$

```

Clave encontrada "Save the planet "

Inciso 5:

a) decrypt(key, *ciph, len) y encrypt(key, *ciph, len)

Están sobre el backend des_compat_* (OpenSSL).

- **Entrada:** key entero con 56 bits efectivos, ciph buffer binario, len múltiplo de 8.
- **Salida:** escriben el resultado en el mismo ciph, usando un buffer temporal para evitar aliasing.

Diagrama:

Key (unit64) → 8 bytes + Paridad DES → DES-Key-Schedule
 Ciph → [bloques de 8 B] → DES-ECB(enc/dec) → tmp → copia a Ciph

b) try_key(key, *ciph, len)

Prueba una clave candidata y decidir si sirve buscando una marca en el texto plano. En este caso la marca es `search_str = " the "`.

Flujo

1. Clona el cifrado a un buffer temporal.
2. Descifra con `decrypt(key, temp, len)`.
3. Convierte el buffer a C-string poniendo `temp[len]=0`.
4. Busca la subcadena con `strstr`.
5. Devuelve 1 si hay match; 0 si no.

Diagrama:

Key $\rightarrow$ `decrypt (Key, ciph)` $\rightarrow$ `temp (texto)`
Temp $\rightarrow$ `strstr (temp, search_str)` $\rightarrow$ ¿encontrado?

c) `memcpy`

Copia exactamente `n` bytes de `src` a `dst` sin preocuparse por el contenido. Es la herramienta correcta cuando no hay solapamiento.

Implementacion

- Copiar cifrado $\rightarrow$ temporal antes de descifrar (`try_key`).
- Copiar bloques de 8B dentro del backend DES.
- Copiar `tmp` $\rightarrow$ `ciph` para simular in-place seguro.

Diagrama:

`memcpy(dst, src, n)`

Si existiera solapamiento se usa `memmove`.

d) `strstr`

Busca la primera aparición de una subcadena en una cadena C.
Retorna puntero a la posición o NULL si no la encuentra.

Es la verificación de que la clave fue correcta: al descifrar, debe aparecer search_str dentro del texto plano.

Diagrama:

```
return strstr(temp, search_str) != NULL;
```

Iniciso 6:

Primitivas MPI:

bruteforce.c reparte el espacio de claves entre n_nodes procesos. Cada proceso prueba su rango y, si acierta, avisa a todos incluido él mismo para que el maestro imprima.

a) MPI_Irecv

Recepción no bloqueante. Inicia la recepción y vuelve de inmediato.

Todos los ranks postean un irecv para estar listos a recibir la clave cuando cualquiera la encuentre.

Codigo:

```
long recv_key = 0;  
MPI_Irecv(&recv_key, 1, MPI_LONG, MPI_ANY_SOURCE, 0, comm, &req);
```

b) MPI_Send

Envío bloqueante. Retorna cuando el runtime ya puede garantizar la entrega buffering o receptor listo.

Patron cuando un proceso encuentra la clave...

Codigo:

```
for (int node = 0; node < n_nodes; ++node) {  
    MPI_Send(&found, 1, MPI_LONG, node, 0, MPI_COMM_WORLD);  
}
```

c) MPI_Wait

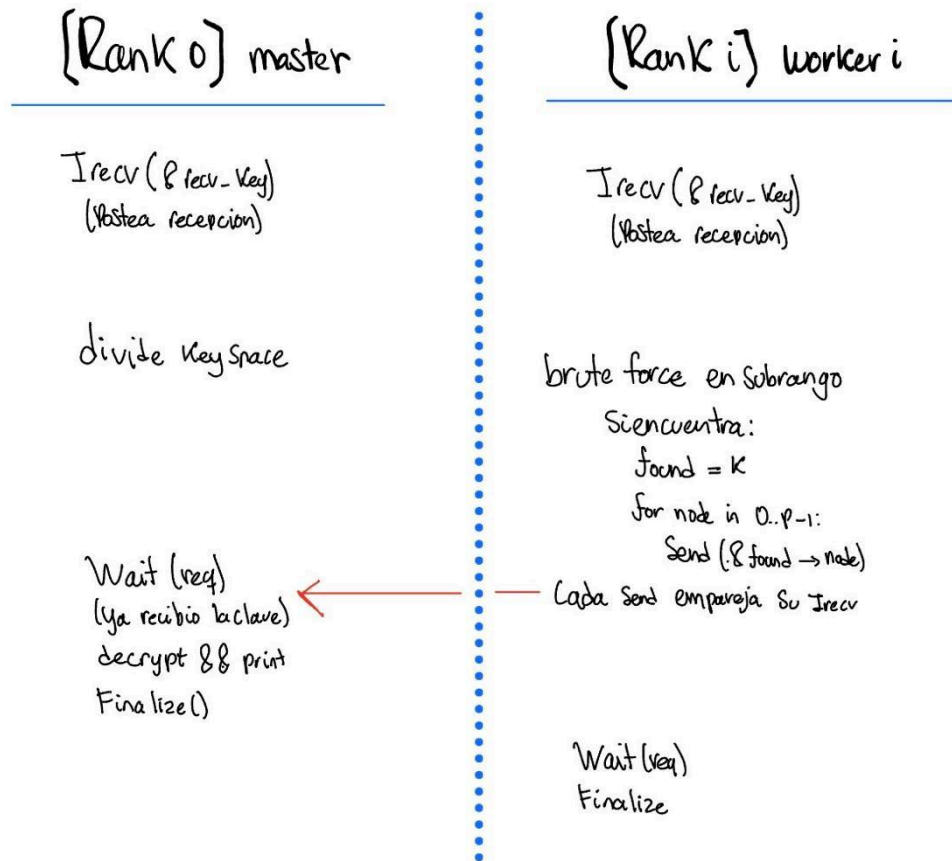
Bloquea hasta que una operación no bloqueante asociada a MPI_Request finalice.

Completar el irecv en todos los ranks antes de finalizar.

Codigo:

```
MPI_Wait(&req, &st);  
if (id == 0) {  
    Imprimir el resultado con la key aca  
}
```

Flujo comunicacion:



Pseudocodigo flujo:

```
MPI_Irecv(&recv_key, 1, MPI_LONG, ANY_SOURCE, 0, comm, &req);

if (encontre) {
    for (node=0..n_nodes-1) MPI_Send(&found, 1, MPI_LONG, node, 0, comm);
}

MPI_Wait(&req, &st);
```

Parte B:

1)

Evidencias:

Texto:

```
Makefile  des_utils.c  bruteforce_seq.c  comandos.txt  seq_sweep.csv  bruteforce_mpi.c  bruteforce.c  run_seq_swee
data > mensaje.txt
1 Solo a Néstor, a Nación Nueva, me llamo la atención cómo la Ó brillaba en el nombre y quedaba grabada en mi memoria.
```

Encriptacion:

```
and@AM-DELL: ~/PY2-Paralela$ ./bin/bruteforce_seq --encrypt -i data/mensaje.txt -k 123456 -o data/cipher.bin
=====
BruteDES • Secuencial
- Cifra y ataca DES-ECB (OpenSSL EVP)
- Rango de llaves configurable [L, U]
=====

✓Cifrado generado
• Entrada : data/mensaje.txt (bytes=121)
• Salida : data/cipher.bin (bytes=128, padded x8)
• Llave : 123456
and@AM-DELL:~/PY2-Paralela$ |
```

Secuencial:

```
and@AM-DELL: ~/PY2-Paralela$ ./bin/bruteforce_seq --bruteforce -c data/cipher.bin -s "me llamo" -L 0 -U 16777216
=====
BruteDES • Secuencial
- Cifra y ataca DES-ECB (OpenSSL EVP)
- Rango de llaves configurable [L, U]
=====

→ BRUTEFORCE SECUENCIAL
• Archivo : data/cipher.bin (bytes=128)
• Subcadena: "me llamo"
• Rango : [0, 16777216)
• Resultado: ✓Llave encontrada
- Llave : 57920
- Texto : Solo a Néstor, a Nación Nueva, me llamo la atención cómo la Ó brillaba en el nombre y quedaba grabada en
mi memoria.
• Tiempo : 0.096546 s
and@AM-DELL:~/PY2-Paralela$ |
```

Paralelo:

```
and@AM-DELL: ~/PY2-Paralela$
• Tiempo : 0.096546 s
and@AM-DELL:~/PY2-Paralela$
and@AM-DELL:~/PY2-Paralela$ mpirun -np 4 ./bin/bruteforce_mpi -c data/cipher.bin -s "me llamo" -L 0 -U 16777216
=====
BruteDES • MPI
- División equitativa del rango y early-stop
=====

→ BRUTEFORCE MPI
• Procesos : 4
• Archivo : data/cipher.bin (bytes=128)
• Subcadena: "me llamo"
• Rango : [0, 16777216)

• Detalle por proceso
RANK | START | END | TESTS | STATUS | TIME(s)
-----|-----|-----|-----|-----|-----
0 | 0 | 4194304 | 57921 | FOUND | 0.0809
1 | 4194304 | 8388608 | 56097 | STOP(SIGNAL) | 0.0788
2 | 8388608 | 12582912 | 57298 | STOP(SIGNAL) | 0.0806
3 | 12582912 | 16777216 | 55265 | STOP(SIGNAL) | 0.0788

• Resultado: ✓ Llave encontrada
- Llave : 57920
- Texto : Solo a Néstor, a Nación Nueva, me llamo la atención cómo la Ó brillaba en el nombre y quedaba grabada en mi memoria.

• Resumen global
- Llaves probadas totales : 226581
- Tiempo total (max rank) : 0.080895 s

and@AM-DELL:~/PY2-Paralela$
```

2)

a) Medir tiempo usando la llave 123456

Encriptacion:

```
and@AM-DELL:~/PY2-Paralela$ ./bin/bruteforce_mpi -s ./data/cipher.bin -c ./des_encrypt -e encrypt
and@AM-DELL:~/PY2-Paralela$ ./bin/bruteforce_seq --encrypt -i data/mensaje.txt -k 123456 -o data/c_123456.bin
=====
BruteDES • Secuencial
- Cifra y ataca DES-ECB (OpenSSL EVP)
- Rango de llaves configurable [L, U)
=====

✓ Cifrado generado
• Entrada : data/mensaje.txt (bytes=32)
• Salida : data/c_123456.bin (bytes=32, padded x8)
• Llave : 123456 (efectiva dec=57920, hex=0x000000000000e240)
```

Solucion:

```

and@AM-DELL:~/PY2-Paralela$ mpirun -np 4 ./bin/bruteforce_mpi -c data/c_123456.bin -s "es una prueba de" -L 0 -U 16777216
6
=====
BruteDES • MPI
- División equitativa del rango y early-stop
=====

→ BRUTEFORCE MPI
• Procesos : 4
• Archivo : data/c_123456.bin (bytes=32)
• Subcadena: "es una prueba de"
• Rango : [0, 16777216)

• Detalle por proceso


| RANK | START    | END      | TESTS | STATUS       | TIME(s) |
|------|----------|----------|-------|--------------|---------|
| 0    | 0        | 4194304  | 57921 | FOUND        | 0.0367  |
| 1    | 4194304  | 8388608  | 59551 | STOP(SIGNAL) | 0.0367  |
| 2    | 8388608  | 12582912 | 59446 | STOP(SIGNAL) | 0.0367  |
| 3    | 12582912 | 16777216 | 58431 | STOP(SIGNAL) | 0.0368  |



• Resultado: ✓ Llave encontrada
- Rank : 0
- Llave : 57920 (efectiva dec=57920, hex=0x000000000000e240)
- Texto : Esta es una prueba de proyecto 2

• Resumen global
- Llaves probadas totales : 235349
- Tiempo total (max rank) : 0.036754 s

```

b) Medir tiempo usando la llave 18014398509481983

Encriptacion:

```

and@AM-DELL: ~/PY2-Paralela$ ./bin/bruteforce_seq --encrypt \
-i data/mensaje.txt \
-k 18014398509481983 \
-o data/cipher_b.bin
=====
BruteDES • Secuencial
- Cifra y ataca DES-ECB (OpenSSL EVP)
- Rango de llaves configurable [L, U)
=====

✓ Cifrado generado
• Entrada : data/mensaje.txt (bytes=32)
• Salida : data/cipher_b.bin (bytes=32, padded x8)
• Llave : 18014398509481983 (efectiva dec=17731819709333246, hex=0x003efefefefefefe)
and@AM-DELL:~/PY2-Paralela$ |

```

Solucion:

```
and@AM-DELL:~/PY2-Paralela$ mpirun -np 4 ./bin/bruteforce_mpi \  
-c data/cipher_b.bin \  
-s "es una prueba de" \  
-L 0 -U 72057594037927936 | tee logs/mpi_case_b_full.txt
```

```
=====
```

```
BruteDES • MPI
```

```
- División equitativa del rango y early-stop
```

```
=====
```

c) Medir tiempo usando la llave 18014398509481984

Encriptacion:

```
and@AM-DELL: ~/PY2-Paralela$ ./bin/bruteforce_seq --encrypt \  
-i data/mensaje.txt \  
-k 18014398509481984 \  
-o data/cipher_c.bin
```

```
=====
```

```
BruteDES • Secuencial
```

```
- Cifra y ataca DES-ECB (OpenSSL EVP)
```

```
- Rango de llaves configurable [L, U)
```

```
=====
```

```
✓Cifrado generado
```

- Entrada : data/mensaje.txt (bytes=32)
- Salida : data/cipher_c.bin (bytes=32, padded x8)
- Llave : 18014398509481984 (efectiva dec=18014398509481984, hex=0x0040000000000000)

```
and@AM-DELL:~/PY2-Paralela$ |
```

Solucion:

```

and@AM-DELL:~/PY2-Paralela$ mpirun -np 4 ./bin/bruteforce_mpi \
-c data/cipher_c.bin \
-s "es una prueba de" \
-L 0 -U 72057594037927936 | tee logs/mpi_case_c_full.txt
=====
BruteDES • MPI
- División equitativa del rango y early-stop
=====

→ BRUTEFORCE MPI
• Procesos : 4
• Archivo : data/cipher_c.bin (bytes=32)
• Subcadena: "es una prueba de"
• Rango : [0, 72057594037927936)

• Detalle por proceso
RANK | START | END | TESTS | STATUS | TIME(s)
-----|-----|-----|-----|-----|-----
0 | 0 | 18014398509481984 | 1 | STOP(SIGNAL) | 0.0012
1 | 18014398509481984 | 36028797018963968 | 1 | FOUND | 0.0010
2 | 36028797018963968 | 54043195528445952 | 1 | STOP(SIGNAL) | 0.0010
3 | 54043195528445952 | 72057594037927936 | 1 | STOP(SIGNAL) | 0.0010

• Resultado: ✓ Llave encontrada
- Rank : 1
- Llave : 18014398509481984 (efectiva dec=18014398509481984, hex=0x0040000000000000)
- Texto : Esta es una prueba de proyecto 2

• Resumen global
- Llaves probadas totales : 4
- Tiempo total (max rank) : 0.001151 s

and@AM-DELL:~/PY2-Paralela$

```

d)

En las pruebas realizadas se observa con claridad que el tiempo de ejecución depende directamente de la posición de la llave dentro del rango y de la forma en que MPI reparte ese rango entre los procesos.

Cuando se utilizó una llave pequeña (123456) dentro de un rango de 2^{24} , esta cayó al inicio del subrango asignado al proceso con rank 0. Como consecuencia, la llave se encontró en milésimas de segundo, ya que ese proceso la probó prácticamente de inmediato.

En el caso contrario, con la llave muy grande del escenario, correspondiente a $(2^{56}/4)-1$, el valor coincidió exactamente con el inicio del subrango asignado al rank 1. Allí ocurrió algo similar: el hallazgo fue instantáneo porque ese proceso la probó en su primer intento, mientras que los demás recibieron la señal y se detuvieron de inmediato.

Sin embargo, el escenario, con la llave $2^{56}/4 - 1$, resultó ser el peor caso. Este valor se ubicó al final del subrango asignado al rank 0, lo que obligó a ese proceso a recorrer prácticamente toda su porción ($\sim 2^{54}$ llaves) antes de encontrarla, volviendo inviable completar la búsqueda en pocos minutos.

En conclusión, bajo las mismas condiciones de hardware y algoritmo, el tiempo de ejecución crece con la “distancia” de la llave respecto al inicio del subrango asignado al proceso que la

contiene. Si la llave aparece justo al comienzo de un subrango, el tiempo de búsqueda es casi nulo; pero si cae al final, el tiempo se dispara.

3)

Alternativas:

Alternativa A Distribución cíclica

Idea: En lugar de asignar un bloque contiguo a cada rank, cada proceso prueba llaves intercaladas. Así, tras t iteraciones, el conjunto de llaves probadas cubre las primeras $t \cdot P$ llaves del rango, reduciendo la varianza y el sesgo por posición de la llave mejor tiempo esperado paralelo.

Alternativa B Master–Worker dinámico con chunks

Idea: Un maestro (rank 0) reparte ventanas de tamaño B a los workers según las van pidiendo. Así:

- Balancea mejor si algunos chunks terminan antes.
- Reduce el tiempo esperado ante posiciones adversas de la llave.
- Mantiene early-stop: cuando alguien encuentra, el maestro envía stop a todos.

Alternativa C permuted-stride

Idea: Cada proceso recorre índices intercalados de la forma $i = \text{rank} + t \cdot P$, pero antes de probar la llave aplica una permutación biyectiva del rango $[L, U)$ mediante un generador lineal congruencial (LCG) con la fórmula $\text{idx} = (A \cdot i + B) \bmod N$. Esto sirve para barajar el orden de prueba y evitar sesgos del orden secuencial. La detección de parada temprana se realiza punto a punto, sin operaciones colectivas dentro del bucle, evitando bloqueos entre procesos.

Alternativa D dinámico adaptativo

Idea: Un proceso maestro reparte bloques de llaves. Cada proceso trabajador solicita trabajo al maestro, quien estima su velocidad de procesamiento y ajusta el tamaño del bloque B para que dure aproximadamente T milisegundos, según el parámetro $-T$. Cuando algún proceso encuentra la llave correcta, el maestro envía una señal de stop a todos los demás. Finalmente, se imprimen las métricas de desempeño por cada proceso y se indica claramente cuál fue el ganador.

4)

Descripción / Pseudocódigo:

Naive (bloques contiguos + early-stop)

dividir $[L, U)$ en P bloques $[L_i, U_i)$

para cada rank i en paralelo:

 Irecv(found)

 para $k = L_i \dots U_i - 1$:

 si Test(Irecv) $\rightarrow$ parar (otro encontró)

 si match(k): found= k ; Send(found) a todos; parar

 fin

recolectar métricas en 0 y reportar

Alternativa A Cíclico

para cada rank i en paralelo:

 Irecv(found)

 para $k = L + i$; $k < U$; $k += P$:

 si Test(Irecv) $\rightarrow$ parar

 si match(k): found= k ; Send(found) a todos; parar

Reporte

Alternativa B Dinámico

MASTER (rank 0):

 next = L

 mientras no found y workers activos:

 si llega FOUND(k): found= k ; responde STOP a todos

 si llega REQ de w :

 si found o next $\geq U$: enviar STOP a w

 sino: enviar TASK[a =next, b =min(next+ B , U)]; next= b

WORKER (rank >0):

 loop:

 enviar REQ

 recibir TASK[a, b] o STOP

 si STOP $\rightarrow$ salir

 para k in [a, b):

 si match(k): enviar FOUND(k) a MASTER; salir

 si Iprobe(STOP) $\rightarrow$ recibir y salir

 reporte

Alternativa C permuted-stride

broadcast(cipher, needle, L , U)

$A \leftarrow$ impar coprimo con $N=U-L$

$B \leftarrow$ seed XOR $f(\text{rank})$

post Irecv(STOP)


```

t0 ← MPI_Wtime()
for t=0.. while not stop:
    if t % K == 0 and Irecv(STOP) completed: break
    i ← rank + t*P
    if i ≥ N: break
    idx ← (A*i + B) mod N
    key ← L + idx
    plain ← DES_decrypt(key, cipher)
    if contains(plain, needle):
        send STOP to all ranks
        winner ← rank; found_key ← key
        break
gather(times, tests, found_flags, keys)
winner ← root picks fastest FOUND
broadcast(winner, found_key)
print tabla por proceso y resultado

```

Alternativa D dinámico adaptativo

```

broadcast(cipher, needle, L, U)
if rank==0 (maestro):
    init next←L, end←U, B[w]≈B0
    para cada worker: recibir REQ y enviar TASK=[a,b) con b-a=B[w]
    loop:
        Probe cualquier mensaje:
        si FOUND(k) desde w*: broadcast STOP, winner←w*, found_key←k, break
        si REQ desde w:
            actualizar throughput[w] con (B[w]/tiempo_chunk)
            B[w] ← clamp(throughput[w] * (T_ms/1000), [Bmin,Bmax])
            asignar TASK siguiente [next, next+B[w])
recolectar (tiempos, tests, status) con Gather
root imprime tabla y resultado (marcando winner)

```

5)

Evidencia pruebas:

logs > bench_full.csv															
mode_cache	bits	U	L	U_run	category	key	P	variant	chunk_B_or_T_or_R	t_seq_s	t_par_s	speedup	rank_found	tests_total	log_file
with_cache	21	2097152	0	2097152	easy	1048577	4	naive		0.679592	0.002415	281.404555	2	104	/home/and/PY2-Paralela/logs/naive_b21_easy_with_cache.txt
with_cache	21	2097152	0	2097152	easy	1048577	4	cyclic		0.679592	0.187288	3.628748	0	1036828	/home/and/PY2-Paralela/logs/cyclic_b21_easy_with_cache.txt
with_cache	21	2097152	0	2097152	easy	1048577	4	dynamic	B=20000	0.679592	0.264894	2.565524	3	1048262	/home/and/PY2-Paralela/logs/dynamic_b21_easy_B20000_with_cache.txt
with_cache	21	2097152	0	2097152	easy	1048577	4	dynamic	B=50000	0.679592	0.286851	2.369146	1	1112709	/home/and/PY2-Paralela/logs/dynamic_b21_easy_B50000_with_cache.txt
with_cache	21	2097152	0	2097152	easy	1048577	4	dynamic	B=100000	0.679592	0.283374	2.398216	2	1062692	/home/and/PY2-Paralela/logs/dynamic_b21_easy_B100000_with_cache.txt
with_cache	21	2097152	0	2097152	easy	1048577	4	dynamic_adaptive	T=10	0.679592	0.238877	2.844945	2	1078781	/home/and/PY2-Paralela/logs/dynamic_adaptive_b21_easy_T10_with_cache.txt
with_cache	21	2097152	0	2097152	easy	1048577	4	dynamic_adaptive	T=30	0.679592	0.231020	2.941782	2	1048577	/home/and/PY2-Paralela/logs/dynamic_adaptive_b21_easy_T30_with_cache.txt
with_cache	21	2097152	0	2097152	easy	1048577	4	dynamic_adaptive	T=50	0.679592	0.311630	2.180766	2	1064832	/home/and/PY2-Paralela/logs/dynamic_adaptive_b21_easy_T50_with_cache.txt
with_cache	21	2097152	0	2097152	easy	1048577	4	permuted	R=12345	0.679592	0.005702	119.184847	3	12658	/home/and/PY2-Paralela/logs/permuted_b21_easy_R12345_with_cache.txt
with_cache	21	2097152	0	2097152	easy	1048577	4	permuted	R=54321	0.679592	0.053344	12.735802	2	299216	/home/and/PY2-Paralela/logs/permuted_b21_easy_R54321_with_cache.txt
with_cache	21	2097152	0	2097152	easy	1048577	4	permuted	R=2025	0.679592	0.007524	90.323232	3	13294	/home/and/PY2-Paralela/logs/permuted_b21_easy_R2025_with_cache.txt
no_cache	21	2097152	0	2097152	easy	1048577	4	naive		0.833047	0.001341	621.213274	2	14	/home/and/PY2-Paralela/logs/naive_b21_easy_no_cache.txt
no_cache	21	2097152	0	2097152	easy	1048577	4	cyclic		0.833047	0.266013	3.131603	1	1041782	/home/and/PY2-Paralela/logs/cyclic_b21_easy_no_cache.txt
no_cache	21	2097152	0	2097152	easy	1048577	4	dynamic	B=20000	0.833047	0.391226	2.129324	2	1037717	/home/and/PY2-Paralela/logs/dynamic_b21_easy_B20000_no_cache.txt
no_cache	21	2097152	0	2097152	easy	1048577	4	dynamic	B=50000	0.833047	0.734263	1.134535	1	1112692	/home/and/PY2-Paralela/logs/dynamic_b21_easy_B50000_no_cache.txt
no_cache	21	2097152	0	2097152	easy	1048577	4	dynamic	B=100000	0.833047	0.623141	1.336852	1	1062693	/home/and/PY2-Paralela/logs/dynamic_b21_easy_B100000_no_cache.txt
no_cache	21	2097152	0	2097152	easy	1048577	4	dynamic_adaptive	T=10	0.833047	0.521680	1.596854	2	1048577	/home/and/PY2-Paralela/logs/dynamic_adaptive_b21_easy_T10_no_cache.txt
no_cache	21	2097152	0	2097152	easy	1048577	4	dynamic_adaptive	T=30	0.833047	0.614751	1.355097	3	1048577	/home/and/PY2-Paralela/logs/dynamic_adaptive_b21_easy_T30_no_cache.txt
no_cache	21	2097152	0	2097152	easy	1048577	4	dynamic_adaptive	T=50	0.833047	0.623491	1.336101	3	1105316	/home/and/PY2-Paralela/logs/dynamic_adaptive_b21_easy_T50_no_cache.txt
no_cache	21	2097152	0	2097152	easy	1048577	4	permuted	R=12345	0.833047	0.006329	131.623795	3	12658	/home/and/PY2-Paralela/logs/permuted_b21_easy_R12345_no_cache.txt
no_cache	21	2097152	0	2097152	easy	1048577	4	permuted	R=54321	0.833047	0.086818	9.684566	0	303125	/home/and/PY2-Paralela/logs/permuted_b21_easy_R54321_no_cache.txt
no_cache	21	2097152	0	2097152	easy	1048577	4	permuted	R=2025	0.833047	0.006603	126.161896	3	13294	/home/and/PY2-Paralela/logs/permuted_b21_easy_R2025_no_cache.txt
with_cache	21	2097152	0	2097152	med	1310720	4	naive		1.430530	0.422950	3.382267	2	1028692	/home/and/PY2-Paralela/logs/naive_b21_med_with_cache.txt
with_cache	21	2097152	0	2097152	med	1310720	4	cyclic		1.430530	0.475802	3.006566	0	1306471	/home/and/PY2-Paralela/logs/cyclic_b21_med_with_cache.txt
with_cache	21	2097152	0	2097152	med	1310720	4	dynamic	B=20000	1.430530	0.673743	2.123258	2	1306213	/home/and/PY2-Paralela/logs/dynamic_b21_med_B20000_with_cache.txt
with_cache	21	2097152	0	2097152	med	1310720	4	dynamic	B=50000	1.430530	0.632160	2.301367	1	1310071	/home/and/PY2-Paralela/logs/dynamic_b21_med_B50000_with_cache.txt

Texto: “Esta es una prueba de proyecto 2”.

Subcadena a buscar: “es una prueba de”.

Bits probados: 21,22,23,24

Llaves por categoría (para cada $U = 2^{\text{bits}}$):

- Easy: $U/2 + 1$
- Med: $U/2 + U/8$
- Hard: $\text{ceil}(U/7) + \text{ceil}(U/13)$

Modos de caché:

- with_cache: ejecución normal.
- no_cache: se limpió la caché del SO antes de correr (`sync; echo 3 > /proc/sys/vm/drop_caches`).

Variantes paralelas todas con MPI, P=4:

- naive bloques contiguos, parada temprana.
- cyclic intercalado $k=L+\text{rank}+t \cdot P$, parada temprana no bloqueante.
- dynamic master–worker, “chunks” fijos -B.
- dynamic_adaptive master–worker que ajusta el chunk al vuelo, objetivo -T ms.
- permuted stride pseudoaleatorio vía LCG, semilla -R para barajar llaves.

Métricas:

- t_seq_s (tiempo secuencial), t_par_s (tiempo paralelo, máximo entre ranks),

- $\text{speedup} = t_{\text{seq_s}} / t_{\text{par_s}}$,
- tests_total (llaves probadas; en dinámico lo sumamos por rank a partir de la tabla).

Resultados:

- En las variantes naive y permuted se observan speedups gigantes cuando la llave cae muy temprano; el early-stop provoca que la ejecución paralela finalice casi al instante, mientras que la secuencial recorrería gran parte del rango.
- Las estrategias cyclic, dynamic y dynamic-adaptive muestran speedups más estables típicamente $2-3\times$ con 4 procesos, ya que distribuyen mejor el trabajo y reducen la influencia del primer acierto.
- Llaves difíciles, cuando la llave aparece cerca del final, el speedup disminuye incluso puede acercarse a $1\times$ si el coste de coordinación y la stop latency anulan la ganancia.
- Efecto del caché: con el caché activado se reducen ligeramente los tiempos y la variabilidad menos I/O frío y módulos de crypto ya cargados. Sin caché, se refleja el coste puro.
- En naive y cíclico suele coincidir con las llaves probadas totales⁰ en dinámicos representa la suma por rank; en permuted depende de la semilla y del orden barajado.
- El tiempo secuencial crece casi linealmente con 2^{bits} . En paralelo también crece, pero amortiguado por el speedup, con llaves tempranas la relación se rompe por el early-stop.
- Tamaño de chunk B (dynamic). Un B grande reduce los mensajes pero aumenta la latencia de parada, un B pequeño mejora la parada pero eleva la sobrecarga. Esto se observa en la dispersión de t_{par} .
- Objetivo T (dynamic-adaptive). Ajusta B automáticamente para estabilizar el throughput; suele ofrecer tiempos consistentes y poca varianza sin necesidad de ajuste manual.
- Semillas en permuted, distintas semillas cambian qué proceso toca primero la llave; al ejecutar varias semillas, los promedios resultan más justos y se reduce el sesgo espacial.
- Con 8 procesos, el algoritmo naive destaca porque las llaves fácil y mediana coinciden justo con los límites de los bloques asignados a cada proceso. Esto permite que el proceso responsable las encuentre casi de inmediato, generando speedups muy altos y tiempos mínimos.
- Con 4 procesos, esa alineación se pierde. La llave mediana queda a mitad de bloque, por lo que el proceso debe recorrer más llaves antes de encontrarla. En este caso, los métodos cíclico y dinámico y los demás reparten mejor la carga y superan al naive.
- Al aumentar los procesos, los bloques se hacen más pequeños. Esto reduce la distancia promedio hasta la llave y mejora especialmente al naive, que depende del early-stop.

Conclusiones:

- Romper DES por fuerza bruta es un problema bastante paralelizable, pero el tiempo real depende mucho de dónde cae la llave dentro del rango y de cómo repartimos el trabajo.

Con la versión naive bloques contiguos el speedup puede ser muy bueno o nulo con cíclico y permutado repartimos mejor la suerte del primer acierto, con dinámico y, sobre todo, dinámico-adaptativo mantenemos el balance incluso si hay ruido o diferencias entre procesos.

- Instrumentar por-rank (tests, tiempo, estado) mas tiempo global (max rank) fue importante para entender quién encontró la llave y por qué hubo o no speedup. El early-stop ahorra trabajo y domina el rendimiento cuando la llave es temprana.
- Caché. Con cache on se repite I/O y datos calientes, y los tiempos bajan ligeramente y con menos variabilidad, con no cache se observa el costo puro.
- La clave decimal que se introduce se convierte a la llave efectiva de 56 bits con paridades OpenSSL ajusta a paridad impar. Por eso a veces ves números como 57920 al recuperar una “123456”.
- Si no se sabe nada de la llave, dinámico-adaptativo suele dar el mejor tiempo esperado y la menor varianza. Cíclico es simple y muy sólido. Permutado reduce sesgos espaciales. Naive es un baseline bueno, pero frágil.
- El naive sobresale con 8 procesos por pura coincidencia geométrica: las llaves caen justo donde empieza su bloque. Con menos procesos, esa ventaja desaparece y los algoritmos que equilibran mejor la carga recuperan el control.
- Los métodos más complejos cíclico, adaptativo, permutado no fallan, solo pierden ventaja cuando el naive tiene suerte con la posición de la llave y el tamaño del bloque.

Recomendaciones:

- Mantener early-stop no bloqueante y la telemetría por proceso.
Ajustar -B tamaño de chunk o usar -T objetivo de ms para estabilizar el balance.
- Medir t_{seq} , t_{par} (max-rank) y tests; calcular speedup como t_{seq} / t_{par} .
- Para 2^{56} , considerar ventanas narrow para campañas de prueba, un barrido full es astronómico.
- Se recomienda usar dynamic-adaptive T 20–50 ms para cargas generales; si se busca simplicidad determinista, optar por cyclic.

Anexo 1

Librerías externas usadas:

- MPI (mpi.h): inicialización y finalización, obtención de rank/P, envíos y recepciones, reducciones, difusiones, versión no-bloqueante en algunas variantes.
- OpenSSL / libcrypto:
 - EVP para cifrar el texto de prueba con DES-ECB en bruteforce_seq .

- DES (legacy) para la vía rápida en el brute force generación de subclaves y DES_ecb_encrypt a nivel bloque.
- C estándar / POSIX: stdint.h, stdio.h, stdlib.h, string.h, time.h, unistd.h, errno.h, getopt.h (parsing de flags), etc

include/des_utils.h / src/des_utils.c

Utilidades de bajo nivel para DES clave de 56 bits efectivos, lectura y escritura y helpers de búsqueda.

void des_encrypt_buffer(uint64_t key56, const unsigned char *in, size_t len, unsigned char *out)

- Entradas
 - key56 (uint64_t): llave entera a 56 bits efectivos los bits de paridad los maneja la lib.
 - in (uchar*), len (size_t): buffer de entrada texto plano, longitud múltiplo de 8 tras padding.
- Salidas
 - out (uchar*): buffer de salida (cifrado).
- Descripción

Cifra in con DES-ECB y escribe en out. Internamente fija paridad de DES y usa una ruta EVP o DES-legacy según se compiló.

void des_decrypt_buffer(uint64_t key56, const unsigned char *in, size_t len, unsigned char *out)

- Entradas: igual que des_encrypt_buffer, pero in es el cifrado.
- Salidas: out queda con el texto plano.
- Descripción: Decifra un bloque múltiplo de 8. Se usa en el brute force, llave por llave.

int contains_substring(const unsigned char *buf, size_t len, const char *needle)

- Entradas
 - buf (uchar*), len (size_t): texto claro candidato.
 - needle (char*): subcadena a buscar.
- Salida

- int (0/1): si la subcadena está presente.
- Descripción: Búsqueda sencilla sobre el texto plano de prueba para validar una llave.

src/bruteforce_seq.c

Binario secuencial: cifra y ataca modo brute force sin MPI.

CLI

- encrypt -i <in.txt> -k <key> -o <cipher.bin>: cifra archivo.
- bruteforce -c <cipher.bin> -s "<substring>" [-L low] [-U up]: busca llave en [L,U).
- Parámetros:
 - -i/-o (ruta), -c (ruta): archivos entrada y salida.
 - -k (uint64): llave decimal o 0xHEX.
 - -s (string): subcadena a validar en texto plano.
 - -L/-U (uint64): límites inferior/superior del rango.
- Salidas por consola
Banners, estado y métricas: llave hallada, texto en claro, y Tiempo: X.XX s.

int main(int argc, char argv)**

- Entradas: flags anteriores.
- Salida: 0 OK, !=0 error.
- Descripción
 - En encrypt: lee -i, aplica padding PKCS#7 múltiplo de 8, cifra con DES-ECB y escribe -o.
 - En bruteforce: carga -c, recorre k=L..U-1, descifra con des_decrypt_buffer(k,...), busca -s. Cronometra y reporta.

src/bruteforce_mpi.c (naive / bloques contiguos)

Divide [L,U) en P bloques contiguos. Cada rank prueba su sub-rango.

int main(int argc, char argv)**

- Entradas
 - c -s -L -U como arriba no hay -B/-T/-R en esta variante.
- Salidas

Tabla detalle por proceso tests y tiempo, resumen global rank ganador, Tiempo total max rank, y texto claro.
- Descripción
 - rank 0 lee -c y difunde (MPI_Bcast) cipher y needle.
 - Cada proceso calcula su bloque:
 $per = (U-L)/P$, $extra = (U-L) \% P$;
 $myL = L + per * rank + \min(rank, extra)$; $myU = myL + per + (rank < extra)$.
 - Bucle $k = myL..myU-1$: descifra y valida.
 - Si alguien encuentra, se hace early-stop reducción / señal no bloqueante + cancelación del bucle local.
 - Se consolida quién encontró y el tiempo máximo entre procesos.

src/bruteforce_mpi_cyclic.c (cíclico / strided)

Intercalado: llave $k = L + rank + t * P$. Mejora el balance temporal de probabilidad de acierto.

- Entradas/Salidas: igual que naive.
- Descripción
 - Difusión inicial MPI_Bcast.
 - Cada rank recorre k en stride P.
 - Early-stop no bloqueante MPI_Iprobe/MPI_Test para minimizar sincronización.
 - Reporta tabla por rank tests, status FOUND/STOP, tiempo.

src/bruteforce_mpi_dynamic.c (dinámico / master-worker)

Maestro reparte chunks de tamaño fijo -B y reasigna conforme los workers se desocupan.

- Entradas (Igual a la naive pero...) adicionales: -B <chunk> (uint64).
- Salidas: Tabla detalle por proceso tests y tiempo, resumen global rank ganador, Tiempo total max rank, y texto claro.

- Descripción
 - rank 0 = master: mantiene next=L y va enviando tramos next, next+B a quien pida, cuando llega a U envía “STOP”.
 - workers: piden trabajo, prueban su chunk y devuelven resultado/estadística.
 - Si un worker encuentra, el master difunde fin inmediato early-stop y corta la cola de trabajo.
 - Tabla por rank tests ejecutados y tiempo, suma de tests = tests_total.

src/bruteforce_mpi_dynamic_adaptive.c (dinámico adaptativo)

Igual al dinámico, pero el master ajusta el tamaño del chunk con un target temporal -T (ms).

- Entradas (Igual a la naive pero...) adicionales: -T <ms> (double).
- Salidas: Tabla detalle por proceso tests y tiempo, resumen global rank ganador, Tiempo total max rank, y texto claro.
- Descripción
 - El master estima throughput de cada worker tests / tiempo y adapta el próximo B para acercarse al tiempo objetivo por asignación T ms.
 - Mantiene muy buen balance incluso con variaciones de rendimiento entre procesos.
 - Imprime tabla por rank con STATUS incluye quién FOUND, tests y tiempo.

src/bruteforce_mpi_permuted.c (stride permutado)

Cada rank recorre llaves con un stride barajado por una permutación lineal (LCG) controlada por -R.

- Entradas (Igual a la naive pero...) adicionales: -R <seed> (uint64).
- Salidas: Tabla detalle por proceso tests y tiempo, resumen global rank ganador, Tiempo total max rank, y texto claro.
- Descripción
 - Se define una biyección $\pi(i) = (a*i + b) \bmod (U-L)$ con a impar, b derivado de seed.
 - El rank r prueba $k = L + \pi(r + t*P)$, esparciendo probabilidades por todo el rango.
 - Early-stop como en cíclico; tabla por rank con FOUND/STOP y tiempos.
 - Útil para evitar sesgos de posición de la llave mejor tiempo paralelo esperado.

Estrategia de paralelización

1. Minimizar la fracción secuencial

- Cifrado y E/S se hacen una vez y se difunden. El trabajo dominante es probar llaves y está totalmente paralelizado.
- Early-stop reduce trabajo inútil post-hallazgo.

2. Balancear la carga

- naive: balance estático puede ser desigual si la llave cae al inicio/fin del bloque.
- cíclico: reparte probabilísticamente el primer acierto entre todos; mejor balance temporal.
- dinámico: balance activo quien termina recibe más muy robusto.
- dinámico adaptativo: balance activo y auto-tuning del chunk al tiempo objetivo.
- permutado: mezcla el orden de prueba y reduce el sesgo espacial mejor expectativa.

3. Minimizar sincronización

- Uso de broadcasts y reducciones puntuales y señales no-bloqueantes para la parada temprana.
- En dinámicos, el control maestro-worker usa mensajes cortos y no requiere barreras.

4. Localidad de datos

- El cifrado y la subcadena se difunden una sola vez; cada worker reusa buffers en caché.
- Los bloques agrupan pruebas contiguas en memoria, lo que ayuda a la CPU con líneas de caché.

¿Qué técnica exacta usa cada variante?

● Naive (bloques contiguos)

- Técnica: partición 1-D estática del rango $[L, U)$ en P bloques.
- La más simple y de baja sobrecarga, pero con alto sesgo si la llave cae muy temprano/tarde en un bloque específico speedup muy variable.

● Cíclica (strided)

- Técnica: partición 1-D intercalada $k = L + \text{rank} + t \cdot P$.
- Balancea la oportunidad de acierto desde el segundo 0; suele mejorar el tiempo paralelo esperado respecto a naive, con prácticamente el mismo coste.

● Dinámica (master-worker con -B)

- Técnica: work stealing centralizado: el master reparte chunks fijos a demanda.
- Elimina desequilibrios y heterogeneidades, si B es razonable, logra buen throughput y baja varianza.

- **Dinámica adaptativa (target -T ms)**

- Técnica: igual a dinámica, ajustando el tamaño de chunk al vuelo con base en rendimiento observado para acercarse a un tiempo objetivo por asignación.
- Muy eficaz cuando hay variabilidad entre procesos, típicamente la de mejor balance.

- **Permutada (stride + LCG con -R)**

- Técnica: cada rank recorre una permutación del índice con stride; evita concentrar llaves tempranas o tardías en un solo proceso.
- Mejora la expectativa respecto a naive, complementa a cíclico mezcla espacio de claves.

Anexo 2

Speedup por variante, caso principal Procesos = 4

Captura de evidencia:

logs > bench_fullcsv																
	node_cache	bits	U	L	U_run	category	key	P	variant	chunk_B_or_T_or_R	t_seq_s	t_par_s	speedup	rank_found	tests_total	log_file
1	with_cache	21	2097152	0	2097152	easy	1048577	4	naive		0.679592	0.002415	281.404555	2	104	/home/and/PV2-Paralela/logs/naive_b21_easy_with_cache.txt
2	with_cache	21	2097152	0	2097152	easy	1048577	4	cyclic		0.679592	0.187280	3.628748	0	1036828	/home/and/PV2-Paralela/logs/cyclic_b21_easy_with_cache.txt
3	with_cache	21	2097152	0	2097152	easy	1048577	4	dynamic	B=20000	0.679592	0.264894	2.565524	3	1048262	/home/and/PV2-Paralela/logs/dynamic_b21_easy_B20000_with_c
4	with_cache	21	2097152	0	2097152	easy	1048577	4	dynamic	B=50000	0.679592	0.286851	2.369146	1	1112709	/home/and/PV2-Paralela/logs/dynamic_b21_easy_B50000_with_c
5	with_cache	21	2097152	0	2097152	easy	1048577	4	dynamic	B=100000	0.679592	0.283374	2.398116	2	1063692	/home/and/PV2-Paralela/logs/dynamic_b21_easy_B100000_with
6	with_cache	21	2097152	0	2097152	easy	1048577	4	dynamic_adaptive	T=10	0.679592	0.238877	2.844945	2	1078781	/home/and/PV2-Paralela/logs/dynamic_adaptive_b21_easy_T10
7	with_cache	21	2097152	0	2097152	easy	1048577	4	dynamic_adaptive	T=30	0.679592	0.231820	2.941702	2	1048577	/home/and/PV2-Paralela/logs/dynamic_adaptive_b21_easy_T30
8	with_cache	21	2097152	0	2097152	easy	1048577	4	dynamic_adaptive	T=50	0.679592	0.311630	2.180766	2	1064832	/home/and/PV2-Paralela/logs/dynamic_adaptive_b21_easy_T50
9	with_cache	21	2097152	0	2097152	easy	1048577	4	permuted	R=12345	0.679592	0.005702	119.184847	3	12658	/home/and/PV2-Paralela/logs/permuted_b21_easy_R12345_with
10	with_cache	21	2097152	0	2097152	easy	1048577	4	permuted	R=54321	0.679592	0.053344	12.739802	2	299216	/home/and/PV2-Paralela/logs/permuted_b21_easy_R54321_with
11	with_cache	21	2097152	0	2097152	easy	1048577	4	permuted	R=2025	0.679592	0.007524	90.323232	3	13294	/home/and/PV2-Paralela/logs/permuted_b21_easy_R2025_with_c
12	no_cache	21	2097152	0	2097152	easy	1048577	4	naive		0.833047	0.001341	621.213274	2	14	/home/and/PV2-Paralela/logs/naive_b21_easy_no_cache.txt
13	no_cache	21	2097152	0	2097152	easy	1048577	4	cyclic		0.833047	0.266013	3.131603	1	1041782	/home/and/PV2-Paralela/logs/cyclic_b21_easy_no_cache.txt
14	no_cache	21	2097152	0	2097152	easy	1048577	4	dynamic	B=20000	0.833047	0.391226	2.129524	2	1037717	/home/and/PV2-Paralela/logs/dynamic_b21_easy_B20000_no_cac
15	no_cache	21	2097152	0	2097152	easy	1048577	4	dynamic	B=50000	0.833047	0.734263	1.134535	1	1112692	/home/and/PV2-Paralela/logs/dynamic_b21_easy_B50000_no_cac
16	no_cache	21	2097152	0	2097152	easy	1048577	4	dynamic	B=100000	0.833047	0.623141	1.336852	1	1062693	/home/and/PV2-Paralela/logs/dynamic_b21_easy_B100000_no_cac
17	no_cache	21	2097152	0	2097152	easy	1048577	4	dynamic_adaptive	T=10	0.833047	0.521680	1.596854	2	1048577	/home/and/PV2-Paralela/logs/dynamic_adaptive_b21_easy_T10
18	no_cache	21	2097152	0	2097152	easy	1048577	4	dynamic_adaptive	T=30	0.833047	0.614751	1.355097	3	1048577	/home/and/PV2-Paralela/logs/dynamic_adaptive_b21_easy_T30
19	no_cache	21	2097152	0	2097152	easy	1048577	4	dynamic_adaptive	T=50	0.833047	0.623491	1.336101	3	1105316	/home/and/PV2-Paralela/logs/dynamic_adaptive_b21_easy_T50
20	no_cache	21	2097152	0	2097152	easy	1048577	4	permuted	R=12345	0.833047	0.006329	131.623795	3	12658	/home/and/PV2-Paralela/logs/permuted_b21_easy_R12345_no_ca
21	no_cache	21	2097152	0	2097152	easy	1048577	4	permuted	R=54321	0.833047	0.086018	9.684566	0	303125	/home/and/PV2-Paralela/logs/permuted_b21_easy_R54321_no_ca
22	no_cache	21	2097152	0	2097152	easy	1048577	4	permuted	R=2025	0.833047	0.006683	126.161896	3	13294	/home/and/PV2-Paralela/logs/permuted_b21_easy_R2025_no_cac
23	with_cache	21	2097152	0	2097152	med	1310720	4	naive		1.430530	0.422950	3.382267	2	1028692	/home/and/PV2-Paralela/logs/naive_b21_med_with_cache.txt
24	with_cache	21	2097152	0	2097152	med	1310720	4	cyclic		1.430530	0.475802	3.006566	0	1306471	/home/and/PV2-Paralela/logs/cyclic_b21_med_with_cache.txt
25	with_cache	21	2097152	0	2097152	med	1310720	4	dynamic	B=20000	1.430530	0.673743	2.123258	2	1306213	/home/and/PV2-Paralela/logs/dynamic_b21_med_B20000_with_ca
26	with_cache	21	2097152	0	2097152	med	1310720	4	dynamic	B=50000	1.430530	0.621600	2.301367	3	1310071	/home/and/PV2-Paralela/logs/dynamic_b21_med_B50000_with_ca
27	with_cache	21	2097152	0	2097152	med	1310720	4	dynamic	B=100000	1.430530	0.758641	1.885648	3	1406559	/home/and/PV2-Paralela/logs/dynamic_b21_med_B100000_with_c
28	with_cache	21	2097152	0	2097152	med	1310720	4	dynamic_adaptive	T=10	1.430530	0.521492	2.743149	2	1318594	/home/and/PV2-Paralela/logs/dynamic_adaptive_b21_med_T10_v
29	with_cache	21	2097152	0	2097152	med	1310720	4	dynamic_adaptive	T=30	1.430530	0.535870	2.479538	3	1333058	/home/and/PV2-Paralela/logs/dynamic_adaptive_b21_med_T30_v
30	with_cache	21	2097152	0	2097152	med	1310720	4	dynamic_adaptive	T=50	1.430530	0.521046	2.745497	3	1310721	/home/and/PV2-Paralela/logs/dynamic_adaptive_b21_med_T50_v
31	with_cache	21	2097152	0	2097152	med	1310720	4	permuted	R=12345	1.430530	0.003826	373.897020	2	16154	/home/and/PV2-Paralela/logs/permuted_b21_med_R12345_with_c
32	with_cache	21	2097152	0	2097152	med	1310720	4	permuted	R=54321	1.430530	0.016591	86.223254	2	80480	/home/and/PV2-Paralela/logs/permuted_b21_med_R54321_with_c
33	with_cache	21	2097152	0	2097152	med	1310720	4	permuted	R=2025	1.430530	0.006567	166.981440	0	29471	/home/and/PV2-Paralela/logs/permuted_b21_med_R2025_with_ca
34	no_cache	21	2097152	0	2097152	med	1310720	4	naive		1.443841	0.385488	3.745489	2	1041856	/home/and/PV2-Paralela/logs/naive_b21_med_no_cache.txt
35	no_cache	21	2097152	0	2097152	med	1310720	4	cyclic		1.443841	0.558921	2.62634	0	1311069	/home/and/PV2-Paralela/logs/cyclic_b21_med_no_cache.txt
36	no_cache	21	2097152	0	2097152	med	1310720	4	dynamic	B=20000	1.443841	0.696931	2.071713	1	1314092	/home/and/PV2-Paralela/logs/dynamic_b21_med_B20000_no_cac
37	no_cache	21	2097152	0	2097152	med	1310720	4	dynamic	B=50000	1.443841	0.605183	0.876023	1	1404844	/home/and/PV2-Paralela/logs/dynamic_b21_med_B50000_no_cac

Captura procesos = 8

logs > bench_full2.csv																
	mode_cache	bits	U	L	U_run	category	key	P	variant	chunk_B_or_T_or_R	t_seq_s	t_par_s	speedup	rank_found	tests_total	log_file
1	with_cache	21	2097152	0	2097152	easy	1048577	8	naive		0.893794	0.001043	856.945350	4	491	/home/and/PV2-Paralela/logs/naive_b21_easy_with_cache.txt
2	with_cache	21	2097152	0	2097152	easy	1048577	8	cyclic		0.893794	0.165919	5.386930	1	1031801	/home/and/PV2-Paralela/logs/cyclic_b21_easy_with_cache.txt
3	with_cache	21	2097152	0	2097152	easy	1048577	8	dynamic	B=20000	0.893794	0.214292	4.170916	2	1044162	/home/and/PV2-Paralela/logs/dynamic_b21_easy_B20000_with_cache.txt
4	with_cache	21	2097152	0	2097152	easy	1048577	8	dynamic	B=50000	0.893794	0.321569	2.779478	2	1280578	/home/and/PV2-Paralela/logs/dynamic_b21_easy_B50000_with_cache.txt
5	with_cache	21	2097152	0	2097152	easy	1048577	8	dynamic	B=100000	0.893794	0.168140	5.315773	3	807725	/home/and/PV2-Paralela/logs/dynamic_b21_easy_B100000_with_cache.txt
6	with_cache	21	2097152	0	2097152	easy	1048577	8	dynamic_adaptive	T=10	0.893794	0.203144	4.399805	1	1048577	/home/and/PV2-Paralela/logs/dynamic_adaptive_b21_easy_T10.txt
7	with_cache	21	2097152	0	2097152	easy	1048577	8	dynamic_adaptive	T=30	0.893794	0.236682	3.776350	1	1066460	/home/and/PV2-Paralela/logs/dynamic_adaptive_b21_easy_T30.txt
8	with_cache	21	2097152	0	2097152	easy	1048577	8	dynamic_adaptive	T=50	0.893794	0.279204	3.201222	2	1101358	/home/and/PV2-Paralela/logs/dynamic_adaptive_b21_easy_T50.txt
9	with_cache	21	2097152	0	2097152	easy	1048577	8	permuted	R=12345	0.893794	0.006746	132.492440	0	55838	/home/and/PV2-Paralela/logs/permuted_b21_easy_R12345_with_cache.txt
10	with_cache	21	2097152	0	2097152	easy	1048577	8	permuted	R=54321	0.893794	0.024803	37.236762	5	206750	/home/and/PV2-Paralela/logs/permuted_b21_easy_R54321_with_cache.txt
11	with_cache	21	2097152	0	2097152	easy	1048577	8	permuted	R=2025	0.893794	0.020139	44.381250	5	106629	/home/and/PV2-Paralela/logs/permuted_b21_easy_R2025_with_cache.txt
12	no_cache	21	2097152	0	2097152	easy	1048577	8	naive		0.993898	0.002080	477.835577	4	103	/home/and/PV2-Paralela/logs/naive_b21_easy_no_cache.txt
13	no_cache	21	2097152	0	2097152	easy	1048577	8	cyclic		0.993898	0.161198	6.165697	1	1030240	/home/and/PV2-Paralela/logs/cyclic_b21_easy_no_cache.txt
14	no_cache	21	2097152	0	2097152	easy	1048577	8	dynamic	B=20000	0.993898	0.309883	3.207333	7	1132878	/home/and/PV2-Paralela/logs/dynamic_b21_easy_B20000_no_cache.txt
15	no_cache	21	2097152	0	2097152	easy	1048577	8	dynamic	B=50000	0.993898	0.309883	3.207333	7	1132878	/home/and/PV2-Paralela/logs/dynamic_b21_easy_B50000_no_cache.txt
16	no_cache	21	2097152	0	2097152	easy	1048577	8	dynamic	B=100000	0.993898	0.320673	3.099413	7	1195931	/home/and/PV2-Paralela/logs/dynamic_b21_easy_B100000_no_cache.txt
17	no_cache	21	2097152	0	2097152	easy	1048577	8	dynamic_adaptive	T=10	0.993898	0.228872	4.342593	2	1083134	/home/and/PV2-Paralela/logs/dynamic_adaptive_b21_easy_T10.txt
18	no_cache	21	2097152	0	2097152	easy	1048577	8	dynamic_adaptive	T=30	0.993898	0.285965	3.475593	2	1102064	/home/and/PV2-Paralela/logs/dynamic_adaptive_b21_easy_T30.txt
19	no_cache	21	2097152	0	2097152	easy	1048577	8	dynamic_adaptive	T=50	0.993898	0.273305	3.636589	1	1151782	/home/and/PV2-Paralela/logs/dynamic_adaptive_b21_easy_T50.txt
20	no_cache	21	2097152	0	2097152	easy	1048577	8	permuted	R=12345	0.993898	0.016792	59.188700	0	72222	/home/and/PV2-Paralela/logs/permuted_b21_easy_R12345_no_cache.txt
21	no_cache	21	2097152	0	2097152	easy	1048577	8	permuted	R=54321	0.993898	0.051045	19.471016	4	275728	/home/and/PV2-Paralela/logs/permuted_b21_easy_R54321_no_cache.txt
22	no_cache	21	2097152	0	2097152	easy	1048577	8	permuted	R=2025	0.993898	0.016873	58.904641	5	110725	/home/and/PV2-Paralela/logs/permuted_b21_easy_R2025_no_cache.txt
23	with_cache	21	2097152	0	2097152	med	1310720	0	naive		1.313747	0.004342	302.567250	5	30	/home/and/PV2-Paralela/logs/naive_b21_med_with_cache.txt
24	with_cache	21	2097152	0	2097152	med	1310720	0	cyclic		1.313747	0.320948	3.993783	0	1283620	/home/and/PV2-Paralela/logs/cyclic_b21_med_with_cache.txt
25	with_cache	21	2097152	0	2097152	med	1310720	8	dynamic	B=20000	1.313747	0.399970	3.284614	6	1335367	/home/and/PV2-Paralela/logs/dynamic_b21_med_B20000_with_cache.txt
26	with_cache	21	2097152	0	2097152	med	1310720	8	dynamic	B=50000	1.313747	0.294520	4.460638	7	1287259	/home/and/PV2-Paralela/logs/dynamic_b21_med_B50000_with_cache.txt
27	with_cache	21	2097152	0	2097152	med	1310720	8	dynamic	B=100000	1.313747	0.306966	4.279700	1	1257714	/home/and/PV2-Paralela/logs/dynamic_b21_med_B100000_with_cache.txt
28	with_cache	21	2097152	0	2097152	med	1310720	8	dynamic_adaptive	T=10	1.313747	0.321266	4.009281	2	1310721	/home/and/PV2-Paralela/logs/dynamic_adaptive_b21_med_T10.txt
29	with_cache	21	2097152	0	2097152	med	1310720	8	dynamic_adaptive	T=30	1.313747	0.282198	4.654009	6	1363544	/home/and/PV2-Paralela/logs/dynamic_adaptive_b21_med_T30.txt
30	with_cache	21	2097152	0	2097152	med	1310720	8	dynamic_adaptive	T=50	1.313747	0.308904	4.252930	3	1310721	/home/and/PV2-Paralela/logs/dynamic_adaptive_b21_med_T50.txt

Las columnas que cuenta el csv son:

- mode_cache: with_cache / no_cache — indica si se limpiaron cachés antes de ejecutar.
- bits: tamaño del espacio de llaves (2^{bits}).
- U: límite superior del espacio (2^{bits}).
- L: límite inferior del rango de búsqueda usado.
- U_run: límite superior del rango realmente recorrido (generalmente =U o ventana local).
- category: dificultad de la posición de la llave: easy, med, hard.
- key: valor entero de la llave verdadera usada para cifrar el texto.
- P: número de procesos MPI.
- variant: algoritmo paralelo: naive, cyclic, dynamic, dynamic_adaptive, permuted.
- chunk_B_or_T_or_R: parámetro de la variante (B=tamaño de chunk, T=objetivo ms, R=semilla).
- t_seq_s: tiempo secuencial (s) medido para el mismo rango.
- t_par_s: tiempo paralelo total (s) (máximo entre ranks).
- speedup: $t_{\text{seq_s}} / t_{\text{par_s}}$.
- rank_found: rank que reportó la llave (vacío si no hubo acierto).
- tests_total: llaves realmente probadas por la ejecución (suma o conteo reportado por la variante).
- log_file: ruta del log de esa corrida.

En total tiene:

- Filas: 264
- Variantes distintas: 5 (cyclic, dynamic, dynamic_adaptive, naive, permuted)

- Casos de bits de 21,22,23,24
- Secuencial: no tiene parámetro extra → 24 filas.
- Naive (MPI simple): no tiene parámetro extra → 24 filas.
- Cíclico: no tiene parámetro extra → 24 filas.
- Dinámico: se ejecuta para cada tamaño de bloque en DYN_B_LIST. Si DYN_B_LIST tiene 3 valores → $24 \times 3 = 72$ filas.
- Dinámico adaptativo: se ejecuta para cada tiempo objetivo en ADAPTIVE_T_LIST. Si ADAPTIVE_T_LIST tiene 3 valores → $24 \times 3 = 72$ filas.
- Permutado: se ejecuta para cada semilla en PERM_SEEDS. Si PERM_SEEDS tiene 3 valores → $24 \times 3 = 72$ filas.

Graficos:

Para Procesos = 4

Sin cache

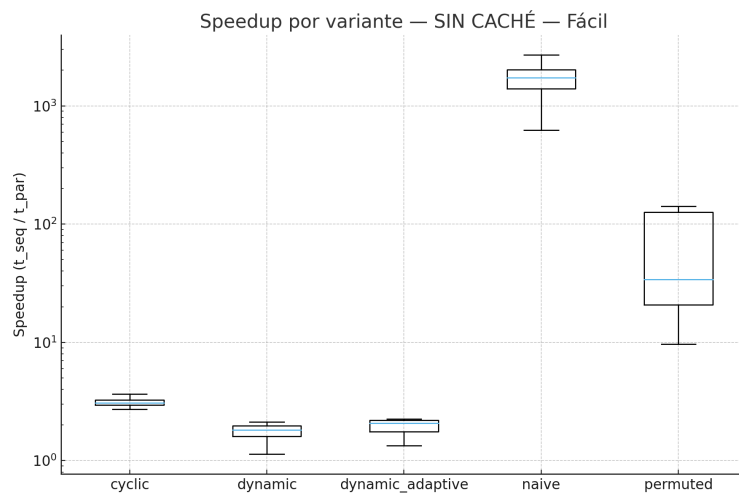


Gráfico 1: speedup sin caché llave facil procesos = 4.

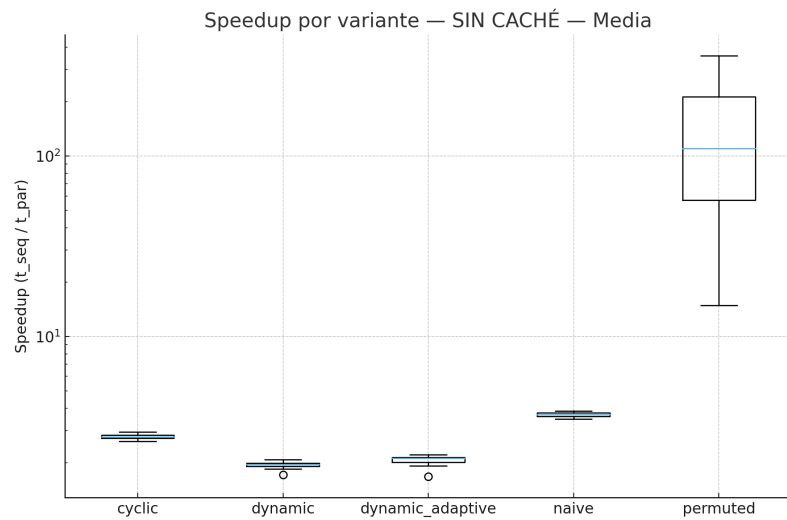


Gráfico 2: speedup sin caché llave media procesos = 4.

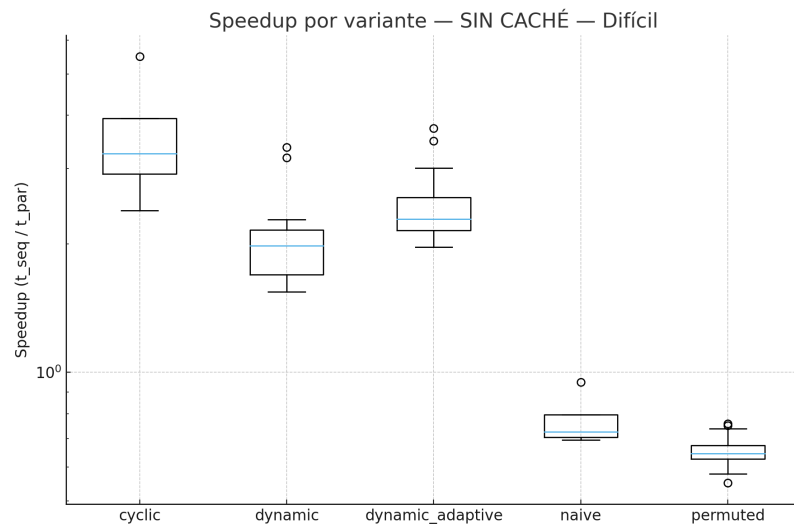


Gráfico 3: speedup sin cache llave difícil procesos = 4.

Con cache

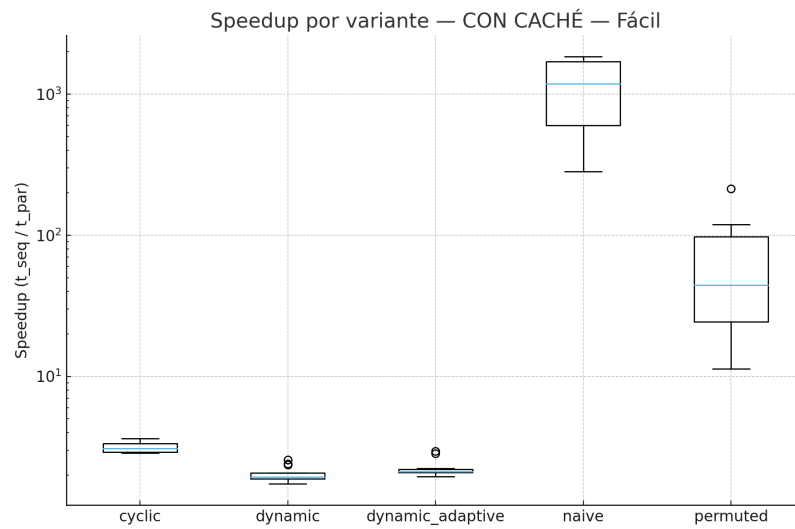


Gráfico 4: speedup con caché llave facil procesos = 4.

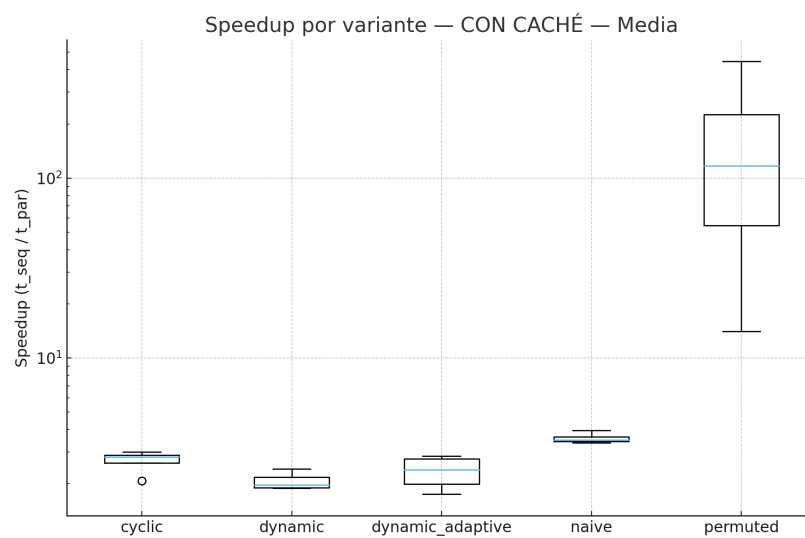


Gráfico 5: speedup con caché llave media procesos = 4.

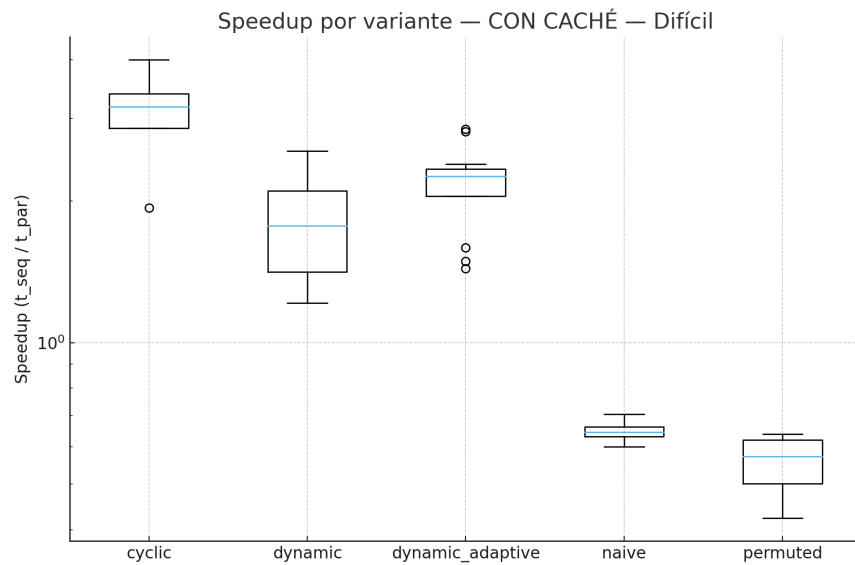


Gráfico 6: speedup con caché llave difícil procesos = 4.

Lo mismo pero para Procesos = 8

Sin cache:

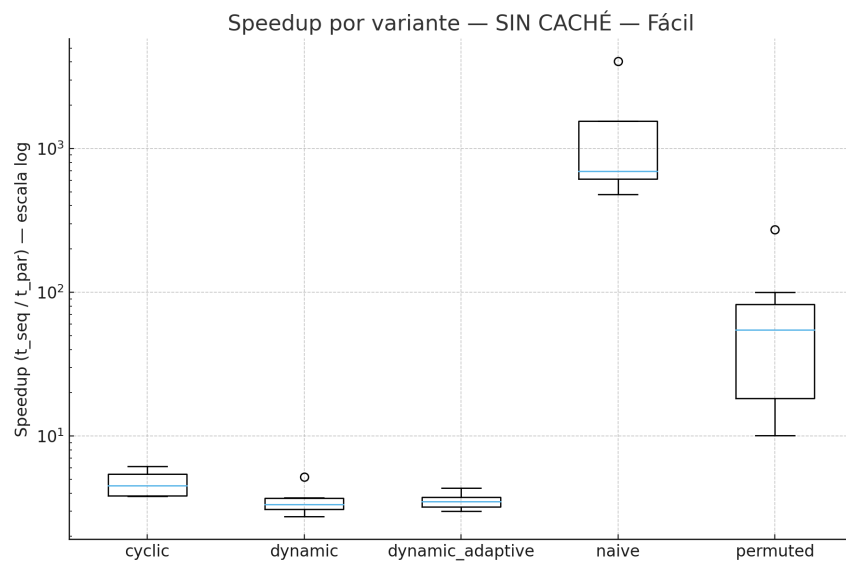


Gráfico 7: speedup con caché llave facil procesos = 8.

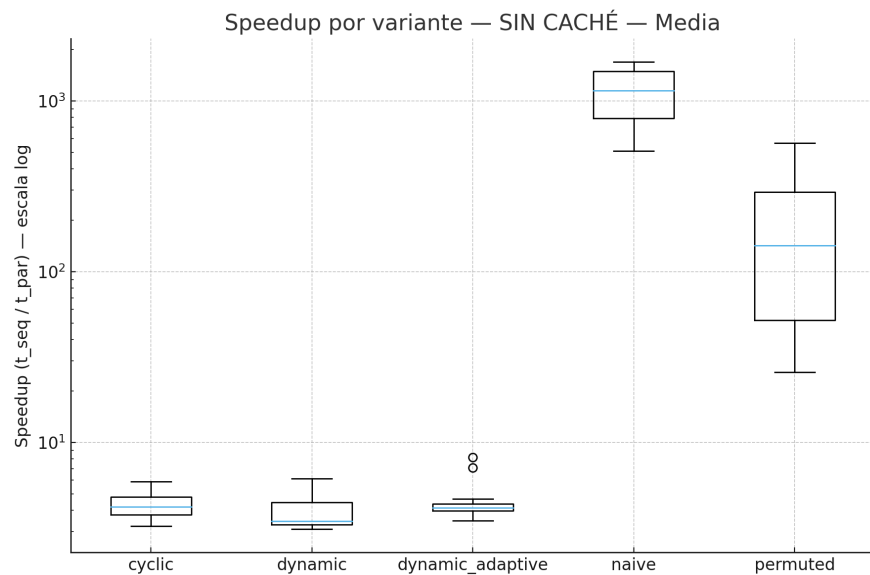


Gráfico 8: speedup con caché llave media procesos = 8.

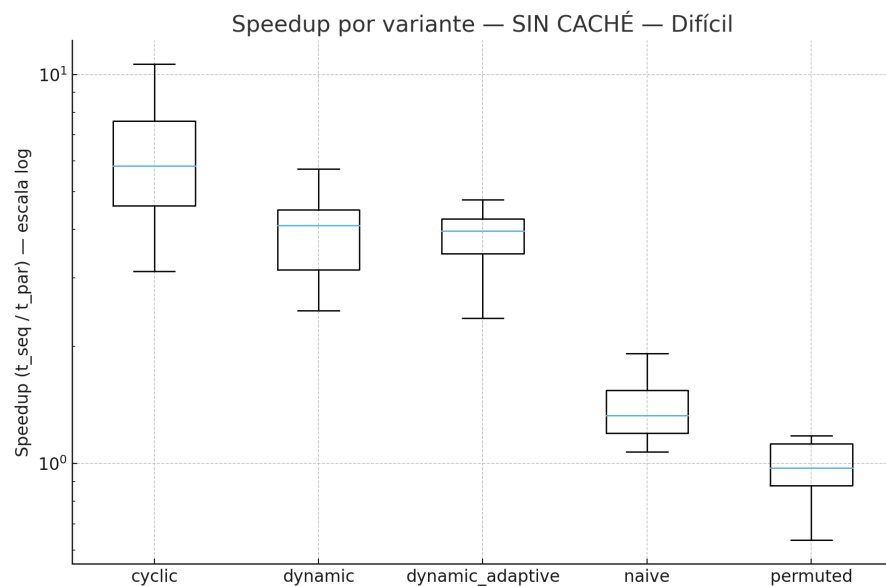


Gráfico 9: speedup con caché llave difícil procesos = 8.

Con cache:

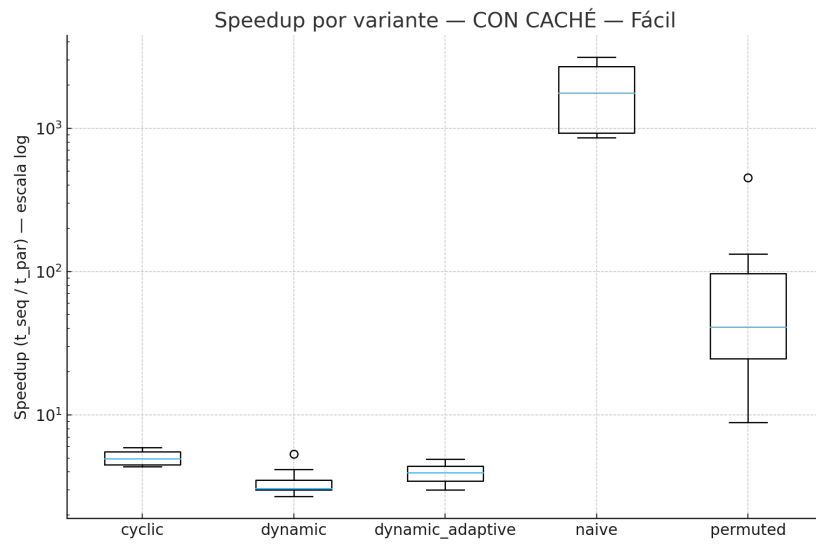


Gráfico 10: speedup con caché llave fácil procesos = 8.

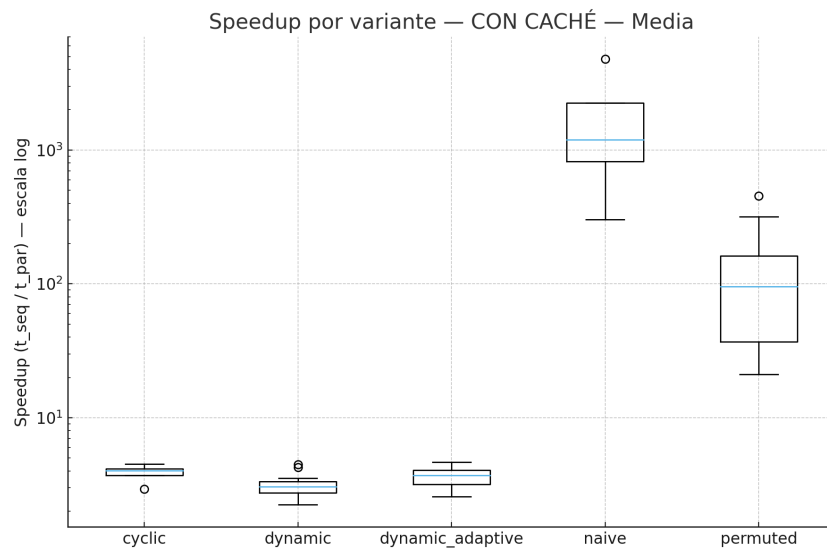


Gráfico 11: speedup con caché llave media procesos = 8.

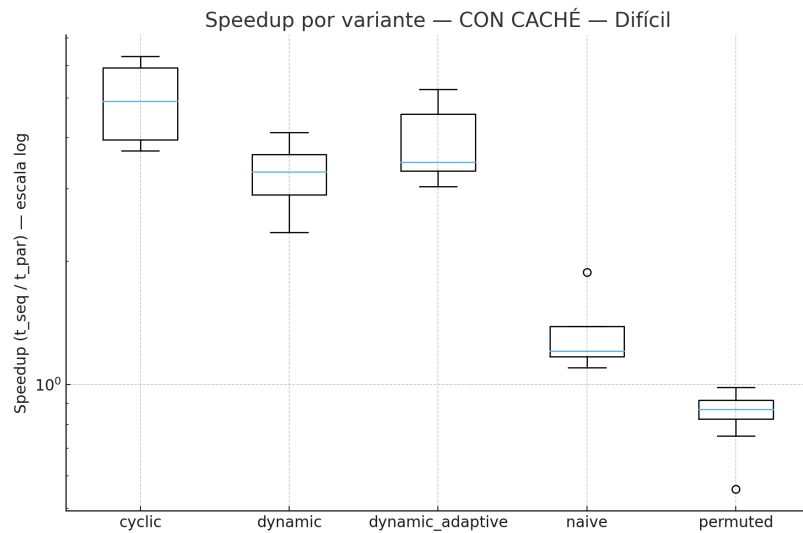


Gráfico 12: speedup con caché llave difícil procesos = 8.

Referencias:

- Msmk. (2024, 10 septiembre). *¿Qué es el DES?* MSMK.
<https://msmk.university/que-es-el-des-msmk-university/>
- IBM Semeru Runtime Certified Edition for z/OS. (2025).
<https://www.ibm.com/docs/es/semeru-runtime-ce-z/21.0.0?topic=differences-data-encryption-standard-des-keys-operations>
- Ibero, J., & Ibero, J. (2022, 12 abril). El cifrado en bloque, ECB Y CBC – IberAsync.es.
IberAsync.es – Blog de tecnología en el mundo IT.
<https://iberasync.es/el-cifrado-en-bloque-ecb-y-cbc/>
- Secret Double Octopus. (2022, 19 octubre). *What is a Brute-force Attack ? - Security Wiki.* <https://doubleoctopus.com/security-wiki/threats-and-tools/brute-force-attack/>

- *How can i parallelize this brute force attack algorithm.* (2019, 2 mayo). Stack Overflow.
<https://stackoverflow.com/questions/55959056/how-can-i-parallelize-this-brute-force-attack-algorithm>